

Employee Management System – Oracle Database Project Report

1. Introduction

This project presents the design and implementation of an Employee Management System using Oracle Database. The system manages departments, employees, salaries, payroll processing, qualifications, leave records, contact information, database views, stored procedures, functions, triggers, and scheduler jobs.

The project demonstrates the use of Oracle SQL and PL/SQL features including Data Definition Language (DDL), Data Manipulation Language (DML), aggregate functions, joins, subqueries, views, procedures, functions, triggers, and automated scheduling.

2. Database Design

The database consists of the following main entities:

- JOB_DEPARTMENT
- EMPLOYEE
- CONTACT_ADD
- SALARY_BONUS
- QUALIFICATION
- REQUIREMENTS
- PAYROLL
- LEAVE

Primary keys, foreign keys, check constraints, and Oracle sequences were implemented to ensure data integrity and automatic key generation.

Implemented Constraints

- Primary Keys for all major entities.
- Foreign Key relationships between parent and child tables.
- CHECK constraint on EMPLOYEE.GENDER allowing only:
 - M
 - F

- CHECK constraint on SALARY_BONUS.AMOUNT ensuring positive salary values.

3. Data Population (DML)

Seed data was inserted successfully into all tables.

Table	Records
JOB_DEPARTMENT	5
EMPLOYEE	10
CONTACT_ADD	10
SALARY_BONUS	5
QUALIFICATION	5
PAYROLL	8
LEAVE	5

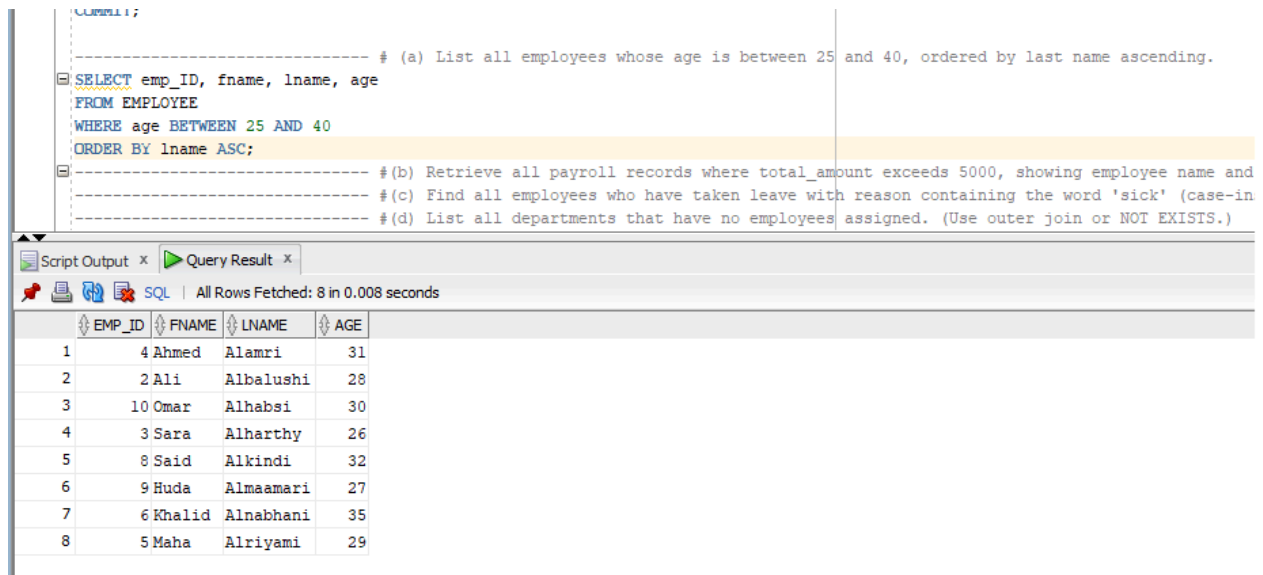
Records Inserted Oracle sequences were used to generate primary key values automatically.

4. Conditional SELECT Queries

Employees Aged Between 25 and 40

A query was executed to retrieve employees aged between 25 and 40 years.

Result: Eight employees satisfied the condition.



The screenshot displays a SQL IDE interface. The top pane shows a SQL script with four comments: (a) List all employees whose age is between 25 and 40, ordered by last name ascending; (b) Retrieve all payroll records where total_amount exceeds 5000, showing employee name and department; (c) Find all employees who have taken leave with reason containing the word 'sick' (case-insensitive); and (d) List all departments that have no employees assigned. (Use outer join or NOT EXISTS.) The script includes a SELECT statement for query (a):

```
SELECT emp_ID, fname, lname, age  
FROM EMPLOYEE  
WHERE age BETWEEN 25 AND 40  
ORDER BY lname ASC;
```


The bottom pane shows the 'Query Result' tab with a table of 8 rows. The table has columns EMP_ID, FNAME, LNAME, and AGE. The data is as follows:

	EMP_ID	FNAME	LNAME	AGE
1	4	Ahmed	Alamri	31
2	2	Ali	Albalushi	28
3	10	Omar	Alhabsi	30
4	3	Sara	Alharthy	26
5	8	Said	Alkindi	32
6	9	Huda	Almaamari	27
7	6	Khalid	Alnabhani	35
8	5	Maha	Alriyami	29

Payroll records with total_amount greater than 5000 were searched together with employee and department information.

Result:

No records were returned because none of the payroll amounts in the sample dataset exceeded 5000.

```
----- # (b) Retrieve all payroll records where total_amount exceeds 5000, showing employee name and department.
SELECT e.fname, e.lname, j.job_dept, p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
  ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT j
  ON p.job_ID = j.job_ID
WHERE p.total_amount > 5000;

----- # (c) Find all employees who have taken leave with reason containing the word 'sick' (case-insensitive).
----- # (d) List all departments that have no employees assigned. (Use outer join or NOT EXISTS.)
```

Script Output x Query Result x

SQL | All Rows Fetched: 0 in 0.014 seconds

FNAME	LNAME	JOB_DEPT	TOTAL_A...
-------	-------	----------	------------

Employees with Sick Leave

A case-insensitive search was performed for leave reasons containing the word "sick".

Result: No matching records were found.

#(c) Find all employees who have taken leave with reason containing the word 'sick' (case-insensitive). I have change to emergency because there is no sick reson.

```

----- # (c) Find all employees who have taken leave with reason
SELECT DISTINCT e.emp_ID, e.fname, e.lname, l.reason
FROM EMPLOYEE e
JOIN LEAVE l
ON e.emp_ID = l.emp_ID
WHERE UPPER(l.reason) LIKE '%EMERGENCY%';

----- # (d) List all departments that have no employees assigned
SELECT j.job_ID, j.job_dept, j.name
FROM JOB_DEPARTMENT j LEFT JOIN EMPLOYEE e
ON j.job_ID = e.job_ID
WHERE e.emp_ID IS NULL;

-----#Write UPDATE statements for the following business scenarios
-----#Give a 10% salary increase to all employees in the 'Engineering' department.
UPDATE SALARY_BONUS
SET amount = amount * 1.10,
    annual = annual * 1.10
WHERE job_ID IN (

```

Query Result x

SQL | All Rows Fetched: 1 in 0.063 seconds

EMP_ID	FNAME	LNAME	REASON
1	4 Ahmed	Alamri	Emergency leave

Departments Without Employees

A LEFT JOIN query was used to identify departments without assigned employees.

Result: No departments without employees were found.

```
-----#(d) List all departments that have n
SELECT j.job_ID, j.job_dept, j.name
FROM JOB_DEPARTMENT j LEFT JOIN EMPLOYEE e
ON j.job_ID = e.job_ID
WHERE e.emp_ID IS NULL;

-----#Write UPDATE statements for the foll
-----#Give a 10% salary increase to all employees in the 'Engineering
UPDATE SALARY_BONUS
SET amount = amount * 1.10,
    annual = annual * 1.10
WHERE job_ID IN (
```

Query Result x

SQL | All Rows Fetched: 0 in 0.007 seconds

JOB_ID	JOB_DEPT	NAME
--------	----------	------

5. UPDATE Operations

Salary Increase

A 10% salary increase was attempted for employees in the Engineering department.

Result: No rows were updated because no Engineering department existed in the dataset .

So I change it to 'IT'

```
-----#Write UPDATE statements for the following business scenarios:
-----#Give a 10% salary increase to all employees in the 'Engineering' department.
UPDATE SALARY_BONUS
SET amount = amount * 1.10,
    annual = annual * 1.10
WHERE job_ID IN (
    SELECT job_ID
    FROM JOB_DEPARTMENT
    WHERE job_dept = 'IT'
);
-----#Update the emp_email of all employees to lowercase using Oracle's LOWER() function.
UPDATE EMPLOYEE
SET emp_email = LOWER(emp_email);
-----#Set the salary_range in JOB_DEPARTMENT to 'REVISED' for any department whose average total payro
UPDATE JOB_DEPARTMENT
```

Query Result x Script Output x

Task completed in 0.092 seconds

1 row updated.

Email Standardization

Employee email addresses were converted to lowercase using Oracle LOWER().

Result: 14 rows were updated successfully.

```
);
-----#Update the emp_email of all employees to lowercase using Oracle's LOWER() function.
UPDATE EMPLOYEE
SET emp_email = LOWER(emp_email);
-----#Set the salary_range in JOB_DEPARTMENT to 'REVISED' for any department whose average total
UPDATE JOB_DEPARTMENT
SET salary_range = 'REVISED'
WHERE job_ID IN (
    SELECT job_ID
    FROM PAYROLL
    GROUP BY job_ID
    HAVING AVG(total amount) > 8000
);
```

Query Result x Script Output x

Task completed in 0.058 seconds

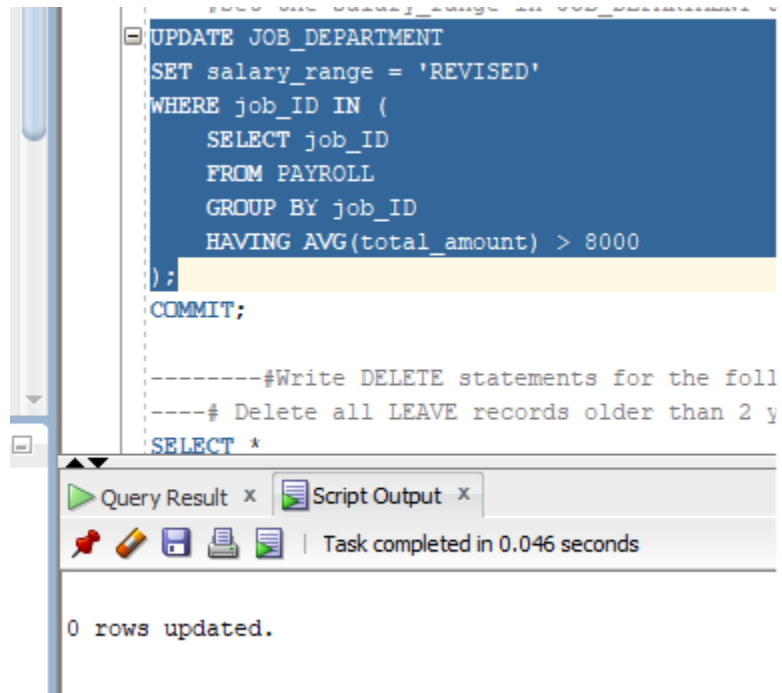
1 row updated.

14 rows updated.

Salary Range Revision

Departments with average payroll amounts greater than 8000 were updated.

Result: No departments met the required condition.



The screenshot shows a SQL IDE window with a script editor and a results pane. The script editor contains the following SQL code:

```
UPDATE JOB_DEPARTMENT
SET salary_range = 'REVISED'
WHERE job_ID IN (
    SELECT job_ID
    FROM PAYROLL
    GROUP BY job_ID
    HAVING AVG(total_amount) > 8000
);
COMMIT;
```

Below the script editor, there are two tabs: "Query Result" and "Script Output". The "Query Result" tab is active and shows the message "0 rows updated." The "Script Output" tab is also visible but empty. The status bar at the bottom indicates "Task completed in 0.046 seconds".

6. DELETE Operations

Old Leave Records

Leave records older than two years were deleted.

Result: No records qualified for deletion.


```

-----#WRITE DELETE statements for the following.
----# Delete all LEAVE records older than 2 years from today.
SELECT *
FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);

----

DELETE FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);

----

SELECT COUNT(*) AS remaining_leave_records
FROM LEAVE;

----# Delete QUALIFICATION records for employees who no longer exi
----# Before executing (b), write a SELECT to preview which record

```

Script Output x Query Result x

SQL | All Rows Fetched: 0 in 0.002 seconds

LEAVE_ID	REASON	LEAVE_D...	EMP_ID	PAYROLL_ID
----------	--------	------------	--------	------------

```
-----#Write DELETE statements for the following:
----# Delete all LEAVE records older than 2 years from today.
SELECT *
FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);
----
DELETE FROM LEAVE
WHERE leave_date < ADD_MONTHS(SYSDATE, -24);
----
SELECT COUNT(*) AS remaining_leave_records
FROM LEAVE;
----# Delete QUALIFICATION records for employees who no longer exist in the E
----# Before executing (b), write a SELECT to preview which records would be (
```

Script Output x Query Result x

Task completed in 0.048 seconds

0 rows deleted.

```
SELECT COUNT(*) AS remaining_leave_records
FROM LEAVE;
----# Delete QUALIFICATION records for employees who no longer exist
----# Before executing (b), write a SELECT to preview which records
```

Script Output x Query Result x

SQL | All Rows Fetched: 1 in 0.002 seconds

REMAINING_LEAVE_RECORDS	
1	5

Orphan Qualification Records

A NOT EXISTS subquery was used to identify qualification records without corresponding employees.

Result: No orphan records were found.

```
FROM LEAVE;
----# Delete QUALIFICATION records for employees who no longer exist in the EMPLOYEE table.
SELECT *
FROM QUALIFICATION q
WHERE NOT EXISTS (
    SELECT 1
    FROM EMPLOYEE e
    WHERE e.emp_ID = q.emp_ID
);
----# Before executing (b), write a SELECT to preview which records would be deleted.
```

Script Output x Query Result x

SQL | All Rows Fetched: 0 in 0.006 seconds

QUAL_ID	POSITION	DATE_IN	EMP_ID
---------	----------	---------	--------

```
);
----
DELETE FROM QUALIFICATION q
WHERE NOT EXISTS (
    SELECT 1
    FROM EMPLOYEE e
    WHERE e.emp_ID = q.emp_ID
);
----
SELECT COUNT(*) AS remaining_qualifications
FROM QUALIFICATION;
----# Before executing (b), write a SELECT to preview which records wo
```

Script Output x Query Result x

Task completed in 0.042 seconds

0 rows deleted.

0 rows deleted.

```
FROM EMPLOYEE e
WHERE e.emp_ID = q.emp_ID
);
-----
SELECT COUNT(*) AS remaining_qualifications
FROM QUALIFICATION;
-----# Before executing (b), write a SELECT to preview which reco
```

Script Output x Query Result x

SQL | All Rows Fetched: 1 in 0.004 seconds

	REMAINING_QUALIFICATIONS
1	5

7. Aggregate Functions

Employee Count Per Department

Worksheet Query Builder

-----Write queries using standard aggregate functions:
 -----(a) Total number of employees in each department.

```

SELECT j.job_dept,
       COUNT(e.emp_ID) AS total_employees
FROM JOB_DEPARTMENT j LEFT JOIN EMPLOYEE e
  ON j.job_ID = e.job_ID
GROUP BY j.job_dept
ORDER BY j.job_dept;
  
```

----- (b) Minimum, maximum, and average salary (amount) across all salary records.

```

SELECT MIN(amount) AS minimum_salary,
       MAX(amount) AS maximum_salary,
       AVG(amount) AS average_salary
FROM SALARY_BONUS;
  
```

----- (c) Total bonus paid out across the entire company.

```

SELECT SUM(bonus) AS total_bonus_paid
FROM SALARY_BONUS;
  
```

Query Result x

SQL | All Rows Fetched: 5 in 0.01 seconds

JOB_DEPT	TOTAL_EMPLOYEES
1 Admin	1
2 Finance	2
3 HR	2
4 IT	3
5 Operations	2

Salary Statistics

```

GROUP BY j.job_dept
ORDER BY j.job_dept;
  
```

----- (b) Minimum, maximum, and average salary (amount) across all salary records.

```

SELECT MIN(amount) AS minimum_salary,
       MAX(amount) AS maximum_salary,
       AVG(amount) AS average_salary
FROM SALARY_BONUS;
  
```

----- (c) Total bonus paid out across the entire company.

```

SELECT SUM(bonus) AS total_bonus_paid
FROM SALARY_BONUS;
  
```

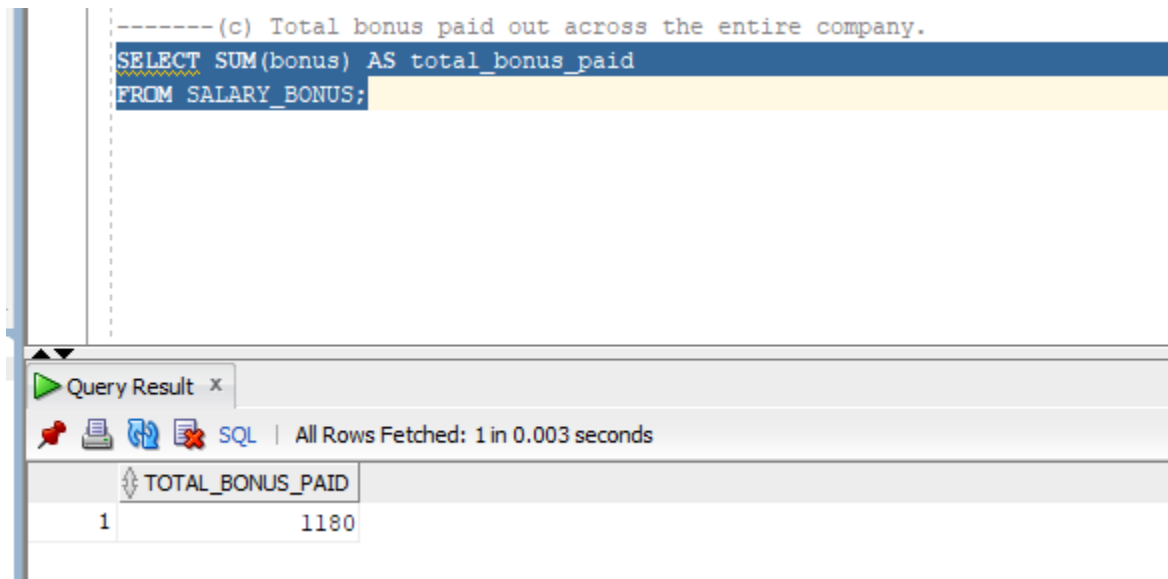
Query Result x

SQL | All Rows Fetched: 1 in 0.005 seconds

	MINIMUM_SALARY	MAXIMUM_SALARY	AVERAGE_SALARY
1	650	1045	839

Total Bonus Paid

Total bonus paid across the company:



The screenshot shows a SQL query editor with the following text:

```
----- (c) Total bonus paid out across the entire company.  
SELECT SUM(bonus) AS total_bonus_paid  
FROM SALARY_BONUS;
```

Below the editor is a 'Query Result' window. It displays the following information:

- Icons for saving, refreshing, and other query actions.
- Text: 'All Rows Fetched: 1 in 0.003 seconds'.
- A table with one column, 'TOTAL_BONUS_PAID', and one row with the value '1180'.

TOTAL_BONUS_PAID
1180

8. GROUP BY and HAVING Queries

Departments with Average Age Greater Than 30

Results:

- Finance
- Operations

```

FROM SALARY_BONUS;
-----Write GROUP BY queries with HAVING filters:
----- (a) List departments where the average employee age exceeds 30.
SELECT j.job_dept,
       AVG(e.age) AS average_age
FROM JOB_DEPARTMENT j
JOIN EMPLOYEE e
  ON j.job_ID = e.job_ID
GROUP BY j.job_dept
HAVING AVG(e.age) > 30;
----- (b) Show all job titles where more than 2 employees share that quali
SELECT q.position,
       COUNT(q.emp_ID) AS total_employees
FROM QUALIFICATION q

```

Query Result x

SQL | All Rows Fetched: 2 in 0.007 seconds

JOB_DEPT	AVERAGE_AGE
1 Finance	31.5
2 Operations	32.5

Qualifications Shared by More Than Two Employees

No qualifications met the requirement.

```

HAVING AVG(e.age) > 30;
----- (b) Show all job titles where more than 2 employees share that qualification position.
SELECT q.position,
       COUNT(q.emp_ID) AS total_employees
FROM QUALIFICATION q
GROUP BY q.position
HAVING COUNT(q.emp_ID) > 2;
----- (c) Find months (from PAYROLL.date) where the total payroll amount exceeds 20,000
SELECT TO_CHAR(payroll_date, 'Month') AS payroll_month,
       SUM(total_amount) AS total_payroll
FROM PAYROLL
GROUP BY TO_CHAR(payroll_date, 'Month')
HAVING SUM(total_amount) > 20000;

```

Query Result x

SQL | All Rows Fetched: 0 in 0.004 seconds

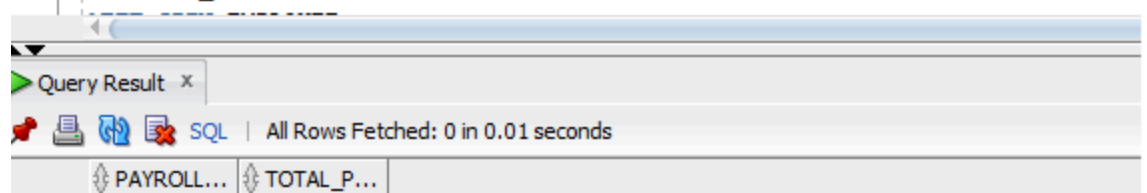
POSITION	TOTAL_E...
----------	------------

Monthly Payroll Totals Above 20,000

No month exceeded the threshold.

```
----- (c) Find months (from PAYROLL.date) where the total payroll amount ex
SELECT TO_CHAR(payroll_date, 'Month') AS payroll_month,
       SUM(total_amount) AS total_payroll
FROM PAYROLL
GROUP BY TO_CHAR(payroll_date, 'Month')
HAVING SUM(total_amount) > 20000;

----- Aggregation with Multiple Functions
---Write a single query that produces a department summary report. For each
--(a) Department name.(b) Total number of employees (COUNT).(c) Total payroll
SELECT j.job_dept AS department_name,
       COUNT(DISTINCT e.emp_ID) AS total_employees,
       SUM(p.total_amount) AS total_payroll_amount,
       AVG(sb.amount) AS average_salary,
       MAX(sb.amount) AS highest_salary,
       MIN(sb.amount) AS lowest_salary
FROM JOB_DEPARTMENT j
```



9. Department Summary Report

A department summary report was generated using:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Findings

The IT department generated the highest payroll total.

The report displayed:

- Department Name

- Total Employees
- Total Payroll Amount
- Average Salary
- Highest Salary
- Lowest Salary

```

HAVING SUM(total_amount) > 20000;
----- Aggregation with Multiple Functions
---Write a single query that produces a department summary report. For each department show:
---(a) Department name.(b) Total number of employees (COUNT).(c) Total payroll amount paid out (SUM of total_amount)
SELECT j.job_dept AS department_name,
       COUNT(DISTINCT e.emp_ID) AS total_employees,
       SUM(p.total_amount) AS total_payroll_amount,
       AVG(sb.amount) AS average_salary,
       MAX(sb.amount) AS highest_salary,
       MIN(sb.amount) AS lowest_salary
FROM JOB_DEPARTMENT j
LEFT JOIN EMPLOYEE e
      ON j.job_ID = e.job_ID
LEFT JOIN PAYROLL p
      ON e.emp_ID = p.emp_ID
LEFT JOIN SALARY_BONUS sb
      ON e.emp_ID = sb.emp_ID
GROUP BY j.job_dept
ORDER BY total_payroll_amount DESC;

```

Query Result x

All Rows Fetched: 5 in 0.014 seconds

	DEPARTMENT_NAME	TOTAL_EMPLOYEES	TOTAL_PAYROLL_AMOUNT	AVERAGE_SALARY	HIGHEST_SALARY	LOWEST_SALARY
1	IT	3	2500	1045	1045	1045
2	Finance	2	2200	850	850	850
3	HR	2	1900	750	750	750
4	Operations	2	1180	900	900	900
5	Admin	1	800	650	650	650

10. INNER JOIN Queries

A complete employee profile was generated using INNER JOIN operations.

Information Retrieved

- Employee ID
- Full Name

- Department
- Qualification
- Salary Amount
- Latest Leave Date

Only employees with both payroll and qualification records were included.

Worksheet Query Builder

```

----Write a query using INNER JOINS to retrieve a complete employee profile:
--Columns: emp_ID, full name (fname || ' ' || lname), department name, job title (from QUALIFICATION), salary amount, latest leave date.
--Filter: only employees who have both a payroll record AND a qualification record.

SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       j.job_dept AS department_name,
       q.position AS job_title,
       sb.amount AS salary_amount,
       MAX(l.leave_date) AS latest_leave_date
FROM EMPLOYEE e INNER JOIN JOB_DEPARTMENT j
  ON e.job_ID = j.job_ID
INNER JOIN QUALIFICATION q
  ON e.emp_ID = q.emp_ID
INNER JOIN PAYROLL p
  ON e.emp_ID = p.emp_ID
INNER JOIN SALARY_BONUS sb
  ON e.emp_ID = sb.emp_ID
LEFT JOIN LEAVE l
  ON e.emp_ID = l.emp_ID
GROUP BY e.emp_ID, e.fname, e.lname, j.job_dept, q.position, sb.amount
ORDER BY e.emp_ID;

```

Query Result x

All Rows Fetched: 5 in 0.113 seconds

EMP_ID	FULL_NAME	DEPARTMENT_NAME	JOB_TITLE	SALARY_AMOUNT	LATEST_LEAVE_DATE
1	Rima Altarshi	IT	Bachelor in IT	1045	05-FEB-26
2	Sara Alharthy	HR	Diploma in HR	750	12-FEB-26
3	Ahmed Alamri	Finance	Bachelor in Accounting	850	01-MAR-26
4	Maha Alriyami	Admin	Business Administration	650	08-MAR-26
5	Khalid Alnabhani	Operations	Operations Management	900	15-MAR-26

11. LEFT OUTER JOIN Queries

Employees Without Leave Records

Five employees were identified as never having taken leave.

```

ORDER BY e.emp_ID,
-----LEFT OUTER JOIN — Missing Records
--Use LEFT OUTER JOINS to find gaps in data:
--(a) List all employees who have never taken any leave (1
SELECT e.emp_ID, e.fname, e.lname
FROM EMPLOYEE e LEFT JOIN LEAVE l
ON e.emp_ID = l.emp_ID
WHERE l.leave_ID IS NULL;
--(b) List all departments that have no salary/bonus reco:

```

Query Result x

SQL | All Rows Fetched: 5 in 0.004 seconds

	EMP_ID	FNAME	LNAME
1	7	Noor	Alfarsi
2	2	Ali	Albalushi
3	8	Said	Alkindi
4	10	Omar	Alhabsi
5	9	Huda	Almaamari

Departments Without Salary Records

No departments lacked salary records.

```

ON e.emp_ID = l.emp_ID
WHERE l.leave_ID IS NULL;
--(b) List all departments that have no salary/bonus records associated with them.
SELECT j.job_ID, j.job_dept, j.name
FROM JOB_DEPARTMENT j LEFT JOIN SALARY_BONUS sb
ON j.job_ID = sb.job_ID
WHERE sb.salary_ID IS NULL;

```

Query Result x

SQL | All Rows Fetched: 0 in 0.007 seconds

JOB_ID	JOB_DEPT	NAME
--------	----------	------

12. Complete Payroll Report

A query joining all major tables produced a consolidated payroll report.

Included Information

- Payroll ID
- Employee Name
- Department
- Qualification
- Salary
- Bonus
- Leave Reason
- Total Payroll Amount

The report was sorted by department and payroll amount.

```
-----Write a single query joining all 6 tables to produce a complete payroll report:
--Columns: payroll_ID, employee full name, department, position (from QUALIFICATION), salary amount, bonus, leave reason, total_amount.
--Order by department name, then by total_amount descending.
SELECT p.payroll_ID, e.fname || ' ' || e.lname AS full_name,
       j.job_dept AS department,
       q.position,
       sb.amount AS salary_amount,
       sb.bonus,
       l.reason AS leave_reason,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
    ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT j
    ON p.job_ID = j.job_ID
JOIN SALARY_BONUS sb
    ON p.salary_ID = sb.salary_ID
LEFT JOIN QUALIFICATION q
    ON e.emp_ID = q.emp_ID
LEFT JOIN LEAVE l
    ON e.emp_ID = l.emp_ID
ORDER BY j.job_dept ASC,
        p.total_amount DESC;
```

PAYROLL_ID	FULL_NAME	DEPARTMENT	POSITION	SALARY_AMOUNT	BONUS	LEAVE_REASON	TOTAL_AMOUNT
1	7 Maha Alriyami	Admin	Business Administration	650	150	Family reason	800
2	5 Ahmed Alamri	Finance	Bachelor in Accounting	850	250	Emergency leave	1100
3	6 Ahmed Alamri	Finance	Bachelor in Accounting	850	250	Emergency leave	1100
4	3 Sara Alharthy	HR	Diploma in HR	750	200	Annual leave	950
5	4 Sara Alharthy	HR	Diploma in HR	750	200	Annual leave	950
6	2 Rima Altarshi	IT	Bachelor in IT	1045	300	Medical leave	1250
7	1 Rima Altarshi	IT	Bachelor in IT	1045	300	Medical leave	1250
8	8 Khalid Alnabhani	Operations	Operations Management	900	280	Annual leave	1180

13. Single-Row Subqueries

Employees Above Average Salary

Employees earning more than the company average salary were identified successfully.

```
-----Write queries using single-row subqueries:
--(a) Find all employees whose salary is greater than the average salary of the entire company.
SELECT e.emp_ID, e.fname, e.lname, sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
  ON e.emp_ID = sb.emp_ID
WHERE sb.amount >
  (
    SELECT AVG(amount)
    FROM SALARY_BONUS
  );
--(b) Retrieve the department with the highest total payroll amount
```

Query Result x

SQL | All Rows Fetched: 3 in 0.007 seconds

EMP_ID	FNAME	LNAME	AMOUNT
1	Rima	Altarshi	1045
2	Ahmed	Alamri	850
3	Khalid	Alnabhani	900

Department with Highest Payroll

The IT department recorded the highest payroll total.

```
--(b) Retrieve the department with the highest total payroll amount
SELECT j.job_ID, j.job_dept,
       SUM(p.total_amount) AS total_payroll
FROM JOB_DEPARTMENT j
JOIN PAYROLL p
  ON j.job_ID = p.job_ID
GROUP BY j.job_ID, j.job_dept
HAVING SUM(p.total_amount) =
(
  SELECT MAX(total_payroll)
  FROM (
    SELECT SUM(total_amount) AS total_payroll
    FROM PAYROLL
    GROUP BY job_ID
  )
);
```

Query Result x

SQL | All Rows Fetched: 1 in 0.011 seconds

JOB_ID	JOB_DEPT	TOTAL_PAYROLL
1	1 IT	2500

14. Multi-Row Subqueries

IN Operator

Employees working in departments with bonus records greater than 500 were retrieved.

```

);
-----Multi-Row Subquery with IN / ANY / ALL Easy 1 pts
--(a) List all employees who work in departments that have at least one salary record with a bonus greater than 500. Use IN.
SELECT emp_ID, fname, lname, job_ID
FROM EMPLOYEE
WHERE job_ID IN (
  SELECT job_ID
  FROM SALARY_BONUS
  WHERE bonus > 500
);
--(b) Find employees whose salary is greater than ALL salaries in the 'Maintenance' department. Use ALL.
--(c) Find employees whose salary is greater than ANY salary in the 'HR' department. Use ANY

```

Query Result x

SQL | All Rows Fetched: 0 in 0.009 seconds

EMP_ID	FNAME	LNAME	JOB_ID
--------	-------	-------	--------

ALL Operator

Employees whose salaries exceeded all salaries in Operations were identified.

```

);
--(b) Find employees whose salary is greater than ALL salaries in the 'Operations' department. Use ALL.
SELECT e.emp_ID, e.fname, e.lname, sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
  ON e.emp_ID = sb.emp_ID
WHERE sb.amount > ALL (
  SELECT sb2.amount
  FROM SALARY_BONUS sb2
  JOIN JOB_DEPARTMENT j
    ON sb2.job_ID = j.job_ID
  WHERE j.job_dept = 'Operations'
);
--(c) Find employees whose salary is greater than ANY salary in the 'HR' department. Use ANY
SELECT e.emp_ID,
       e.fname,
       e.lname,
       sb.amount
FROM EMPLOYEE e

```

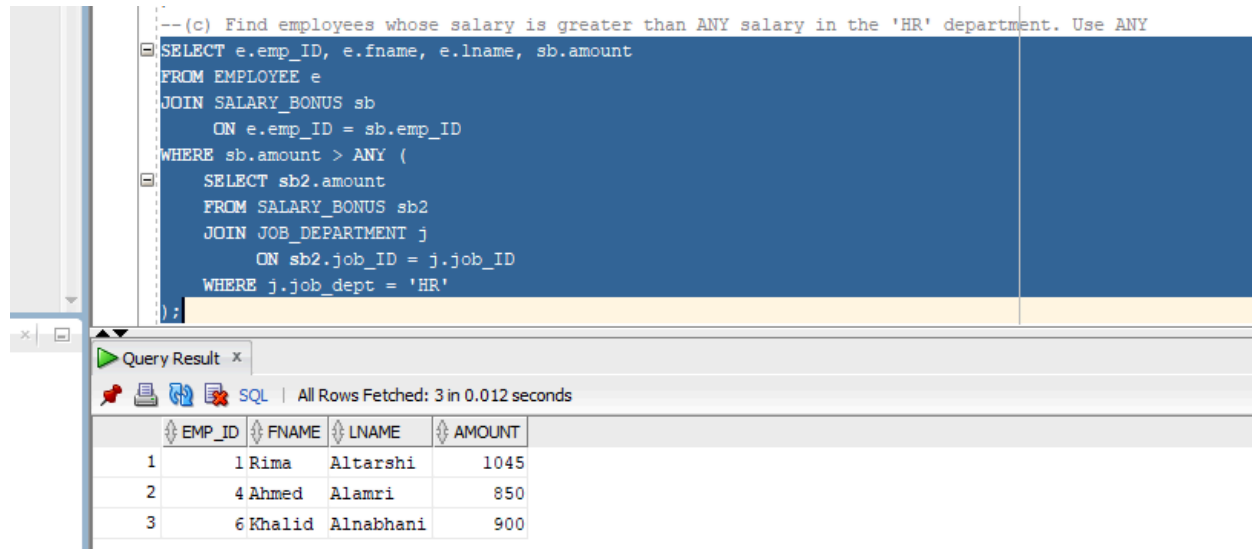
Query Result x

SQL | All Rows Fetched: 1 in 0.034 seconds

EMP_ID	FNAME	LNAME	AMOUNT
1	Rima	Altarshi	1045

ANY Operator

Employees whose salaries exceeded at least one salary in HR were returned successfully.



```
--(c) Find employees whose salary is greater than ANY salary in the 'HR' department. Use ANY
SELECT e.emp_ID, e.fname, e.lname, sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
  ON e.emp_ID = sb.emp_ID
WHERE sb.amount > ANY (
  SELECT sb2.amount
  FROM SALARY_BONUS sb2
  JOIN JOB_DEPARTMENT j
    ON sb2.job_ID = j.job_ID
  WHERE j.job_dept = 'HR'
);
```

Query Result x

SQL | All Rows Fetched: 3 in 0.012 seconds

EMP_ID	FNAME	LNAME	AMOUNT
1	Rima	Altarshi	1045
2	Ahmed	Alamri	850
3	Khalid	Alnabhani	900

15. Views

VW_EMPLOYEE_SUMMARY

Displays:

- Employee ID
- Full Name
- Gender
- Age
- Department
- Qualification

Female Employees Over 30

The view was queried successfully.


```
LEFT JOIN QUALIFICATION q
  ON e.emp_ID = q.emp_ID;
--After creating it:
--(a) Query the view to list all female employees over 30.
SELECT *
FROM VW_EMPLOYEE_SUMMARY
WHERE gender = 'F'
  AND age > 30;
--(b) Try to INSERT a row through this view and document the Oracle error message you receive
```

Script Output x Query Result x

SQL | All Rows Fetched: 0 in 0.012 seconds

EMP_ID	FULL_NAME	GENDER	AGE	DEPARTM...	JOB_TITLE
--------	-----------	--------	-----	------------	-----------

Insert Attempt

Oracle returned:

ORA-01776

because the view is based on multiple joined tables and is therefore not directly updatable.

The view is based on multiple tables using JOIN operations. Oracle cannot determine which base table should receive the inserted data, so the INSERT operation is not allowed and returns ORA-01776.

```
JOIN JOB_DEPARTMENT j
ON e.job_ID = j.job_ID
LEFT JOIN QUALIFICATION q
ON e.emp_ID = q.emp_ID;
--After creating it:
--(a) Query the view to list all female employees over 30.
SELECT
FROM VW_EMPLOYEE_SUMMARY
WHERE gender = 'F'
AND age > 30;
--(b) Try to INSERT a row through this view and document the Oracle error message you receive
INSERT INTO VW_EMPLOYEE_SUMMARY (emp_ID, full_name, gender, age, department_name, job_title)VALUES (999, 'Test User', 'F', 31, 'IT', 'Test Position');
```

Script Output x Query Result x

Task completed in 0.067 seconds

*Cause: A database operation was attempted without the required privilege(s).

*Action: Ask your database administrator or security administrator to grant you the required privilege(s).

View VW_EMPLOYEE_SUMMARY created.

Error starting at line : 21 in command -
INSERT INTO VW_EMPLOYEE_SUMMARY (emp_ID, full_name, gender, age, department_name, job_title)VALUES (999, 'Test User', 'F', 31, 'IT', 'Test Position')

Error at Command Line : 21 Column : 34

Error report -
SQL Error: ORA-01779: cannot modify a column which maps to a non key-preserved table

<https://docs.oracle.com/error-help/db/ora-01779/01779.00000> - "cannot modify a column which maps to a non key-preserved table"

*Cause: An attempt was made to insert or update columns of a join view which map to a non-key-preserved table.

*Action: Modify the underlying base tables directly.

More Details :
<https://docs.oracle.com/error-help/db/ora-01779/>

VW_PAYROLL_DASHBOARD

Displays:

- Payroll ID
- Employee Name
- Department
- Salary
- Bonus
- Leave Reason
- Payroll Date
- Total Amount

Top 5 Payroll Records

The view successfully returned the five highest payroll records ordered by total amount.

<pre> p.total_amount FROM PAYROLL p JOIN EMPLOYEE e ON p.emp_ID = e.emp_ID JOIN JOB_DEPARTMENT j ON p.job_ID = j.job_ID JOIN SALARY_BONUS sb ON p.salary_ID = sb.salary_ID LEFT JOIN LEAVE l ON p.emp_ID = l.emp_ID; --Query the view to find the top 5 payroll records by total_amount SELECT * FROM VW_PAYROLL_DASHBOARD ORDER BY total_amount DESC FETCH FIRST 5 ROWS ONLY; </pre>																																																							
Script Output x Query Result x All Rows Fetched: 5 in 0.037 seconds <table> <tr> <th>PAYROLL_ID</th><th>FULL_NAME</th><th>DEPARTMENT</th><th>SALARY_AMOUNT</th><th>BONUS</th><th>LEAVE_REASON</th><th>PAYROLL_DATE</th><th>TOTAL_AMOUNT</th></tr> <tr> <td>1</td><td>Rima Altarshi</td><td>IT</td><td>1045</td><td>300</td><td>Medical leave</td><td>31-JAN-26</td><td>1250</td></tr> <tr> <td>2</td><td>Rima Altarshi</td><td>IT</td><td>1045</td><td>300</td><td>Medical leave</td><td>28-FEB-26</td><td>1250</td></tr> <tr> <td>3</td><td>Khalid Alnabhani</td><td>Operations</td><td>900</td><td>280</td><td>Annual leave</td><td>31-JAN-26</td><td>1180</td></tr> <tr> <td>4</td><td>Ahmed Alamri</td><td>Finance</td><td>850</td><td>250</td><td>Emergency leave</td><td>31-JAN-26</td><td>1100</td></tr> <tr> <td>5</td><td>Ahmed Alamri</td><td>Finance</td><td>850</td><td>250</td><td>Emergency leave</td><td>28-FEB-26</td><td>1100</td></tr> </table>								PAYROLL_ID	FULL_NAME	DEPARTMENT	SALARY_AMOUNT	BONUS	LEAVE_REASON	PAYROLL_DATE	TOTAL_AMOUNT	1	Rima Altarshi	IT	1045	300	Medical leave	31-JAN-26	1250	2	Rima Altarshi	IT	1045	300	Medical leave	28-FEB-26	1250	3	Khalid Alnabhani	Operations	900	280	Annual leave	31-JAN-26	1180	4	Ahmed Alamri	Finance	850	250	Emergency leave	31-JAN-26	1100	5	Ahmed Alamri	Finance	850	250	Emergency leave	28-FEB-26	1100
PAYROLL_ID	FULL_NAME	DEPARTMENT	SALARY_AMOUNT	BONUS	LEAVE_REASON	PAYROLL_DATE	TOTAL_AMOUNT																																																
1	Rima Altarshi	IT	1045	300	Medical leave	31-JAN-26	1250																																																
2	Rima Altarshi	IT	1045	300	Medical leave	28-FEB-26	1250																																																
3	Khalid Alnabhani	Operations	900	280	Annual leave	31-JAN-26	1180																																																
4	Ahmed Alamri	Finance	850	250	Emergency leave	31-JAN-26	1100																																																
5	Ahmed Alamri	Finance	850	250	Emergency leave	28-FEB-26	1100																																																

16. Stored Procedure

SP_ADD_EMPLOYEE

The procedure:

- Validates email uniqueness.
- Prevents duplicate emails.
- Inserts employee data.
- Displays confirmation using DBMS_OUTPUT.

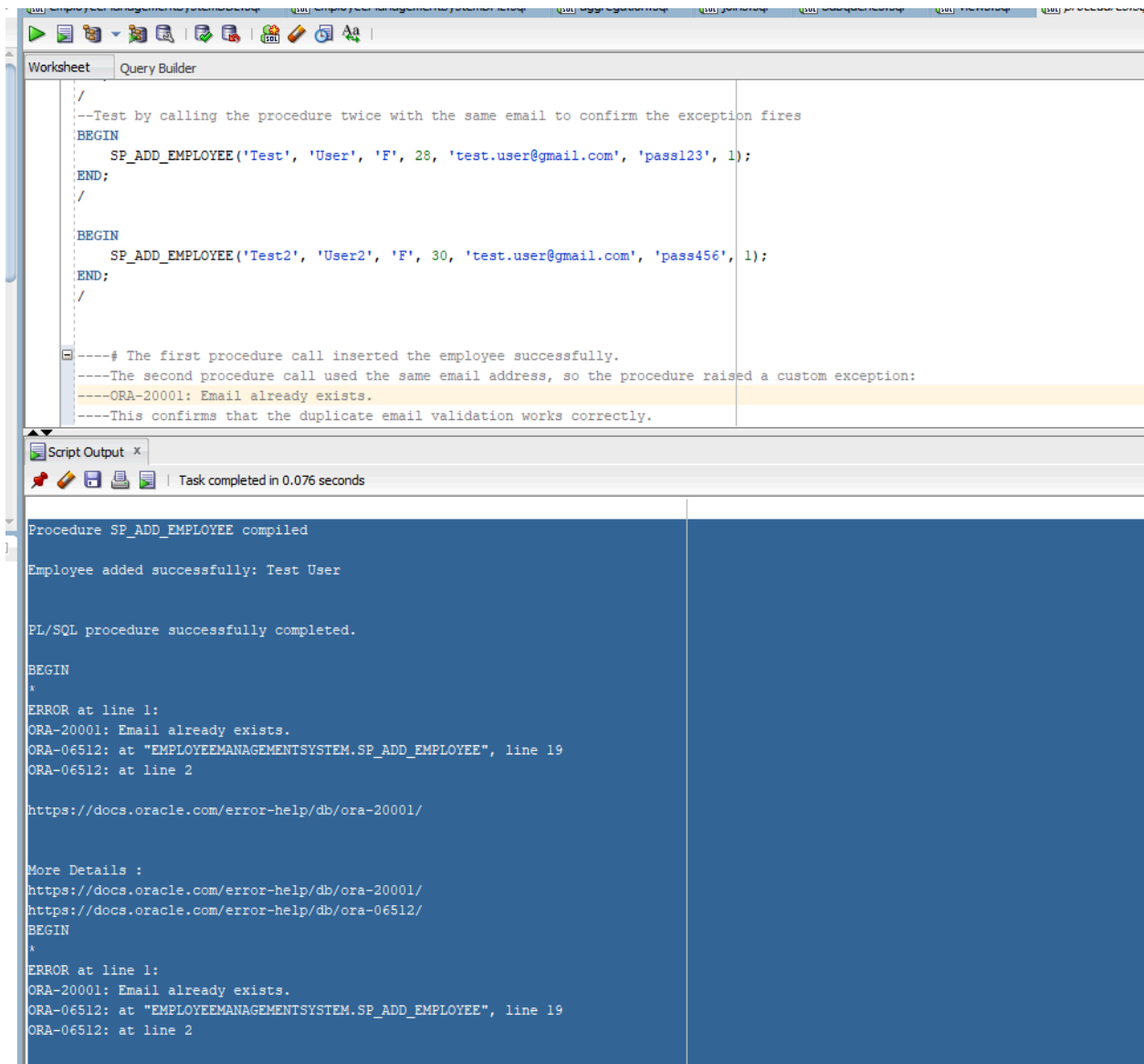
Test Results

The first procedure call inserted the employee successfully.

The second procedure call used the same email address, so the procedure raised a custom exception:

ORA-20001: Email already exists.

This confirms that the duplicate email validation works correctly.



The screenshot displays the Oracle SQL Developer environment. The top pane, titled 'Worksheet', contains a PL/SQL script designed to test a custom exception. The script consists of two blocks, each starting with 'BEGIN' and ending with 'END;'. The first block calls the procedure `SP_ADD_EMPLOYEE` with parameters ('Test', 'User', 'F', 28, 'test.user@gmail.com', 'pass123', 1). The second block calls the same procedure with parameters ('Test2', 'User2', 'F', 30, 'test.user@gmail.com', 'pass456', 1). Below the script, a comment block explains the test: the first call is successful, the second call uses the same email address and thus raises a custom exception, which is identified as ORA-20001: Email already exists. The bottom pane, titled 'Script Output', shows the execution results. It confirms that the procedure was compiled, the first employee was added successfully, and the PL/SQL procedure completed. It then displays the error for the second call: 'ERROR at line 1: ORA-20001: Email already exists.' and 'ORA-06512: at "EMPLOYEEMANAGEMENTSYSTEM.SP_ADD_EMPLOYEE", line 19'. It also provides a link to the Oracle error documentation for ORA-20001.

```
--Test by calling the procedure twice with the same email to confirm the exception fires
BEGIN
  SP_ADD_EMPLOYEE('Test', 'User', 'F', 28, 'test.user@gmail.com', 'pass123', 1);
END;
/

BEGIN
  SP_ADD_EMPLOYEE('Test2', 'User2', 'F', 30, 'test.user@gmail.com', 'pass456', 1);
END;
/

-----# The first procedure call inserted the employee successfully.
-----The second procedure call used the same email address, so the procedure raised a custom exception:
-----ORA-20001: Email already exists.
-----This confirms that the duplicate email validation works correctly.
```

Script Output x

Task completed in 0.076 seconds

Procedure SP_ADD_EMPLOYEE compiled

Employee added successfully: Test User

PL/SQL procedure successfully completed.

BEGIN

*

ERROR at line 1:

ORA-20001: Email already exists.

ORA-06512: at "EMPLOYEEMANAGEMENTSYSTEM.SP_ADD_EMPLOYEE", line 19

ORA-06512: at line 2

<https://docs.oracle.com/error-help/db/ora-20001/>

More Details :

<https://docs.oracle.com/error-help/db/ora-20001/>

<https://docs.oracle.com/error-help/db/ora-06512/>

BEGIN

*

ERROR at line 1:

ORA-20001: Email already exists.

ORA-06512: at "EMPLOYEEMANAGEMENTSYSTEM.SP_ADD_EMPLOYEE", line 19

ORA-06512: at line 2

17. Function

FN_NET_SALARY

Calculates:

Net Salary = Salary + Bonus

Result

Employees were listed with their calculated net salary and ordered from highest to lowest.

```
SELECT NVL(amount,0) + NVL(bonus,0)
  INTO v_net_salary
  FROM SALARY_BONUS
  WHERE emp_ID = p_emp_ID;

RETURN v_net_salary;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 0;
END;
```

```
--SELECT
SELECT FN_NET_SALARY(1)
FROM DUAL;
---#SELECT salary(amount + bonus)
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       FN_NET_SALARY(e.emp_ID) AS net_salary
FROM EMPLOYEE e
ORDER BY net_salary DESC;
```

Script Output x

Task completed in 0.187 seconds

Function FN_NET_SALARY compiled

Worksheet Query Builder

--Select a function named FN_NET_SALARY that accepts emp_ID as input and returns the employee's net salary.
 --Use it in a SELECT query to display all employees with their calculated net salary, ordered highest to lowest.

```

CREATE OR REPLACE FUNCTION FN_NET_SALARY (
  p_emp_ID IN NUMBER
)
RETURN NUMBER
IS
  v_net_salary NUMBER;
BEGIN
  SELECT NVL(amount,0) + NVL(bonus,0)
  INTO v_net_salary
  FROM SALARY_BONUS
  WHERE emp_ID = p_emp_ID;

  RETURN v_net_salary;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 0;
END;
/

--SELECT
SELECT FN_NET_SALARY(1)
FROM DUAL;
---#SELECT salary(amount + bonus)
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       FN_NET_SALARY(e.emp_ID) AS net_salary
FROM EMPLOYEE e
ORDER BY net_salary DESC;
  
```

Script Output x Query Result x

SQL | All Rows Fetched: 11 in 0.013 seconds

EMP_ID	FULL_NAME	NET_SALARY
1	1 Rima Altarshi	1345
2	6 Khalid Alnabhani	1180
3	4 Ahmed Alamri	1100
4	3 Sara Alharthy	950
5	5 Maha Alriyami	800
6	9 Huda Almaamari	0
7	10 Omar Alhabbsi	0
8	8 Said Alkindi	0
9	2 Ali Albalushi	0
10	11 Test User	0
11	7 Noor Alfarsi	0

18. Triggers

TRG_EMP_ID

A BEFORE INSERT trigger automatically generates employee IDs using a sequence.

Result

Employees can be inserted without manually specifying emp_ID.

```

INCREMENT BY 1;
--If the inserted emp_ID is NULL, the trigger should populate it from a sequence (EMP_SEQ.NEXTVAL).
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF :NEW.emp_ID IS NULL THEN
        :NEW.emp_ID := EMP_SEQ.NEXTVAL;
    END IF;
END;

--Test: INSERT an employee without specifying emp_ID and verify it was auto-assigned
INSERT INTO EMPLOYEE (
    fname, lname, gender, age, emp_email, emp_pass, job_ID
)
VALUES (
    'Lina', 'Alharthy', 'F', 27, 'lina@gmail.com', 'pass123', 1
);

COMMIT;
SELECT emp_ID, fname, lname, emp_email
FROM EMPLOYEE
WHERE emp_email = 'lina@gmail.com';

```

Script Output x

Task completed in 0.054 seconds

Sequence EMP_SEQ created.

Trigger TRG_EMP_ID compiled

The screenshot shows a SQL Query Builder window with the following SQL code:

```

---Create a BEFORE INSERT trigger named TRG_EMP_ID on the EMPLOYEE table.
CREATE SEQUENCE EMP_SEQ
START WITH 1
INCREMENT BY 1;
--If the inserted emp_ID is NULL, the trigger should populate it from a sequence (EMP_SEQ.NEXTVAL).
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF :NEW.emp_ID IS NULL THEN
        :NEW.emp_ID := EMP_SEQ.NEXTVAL;
    END IF;
END;
/

--Test: INSERT an employee without specifying emp_ID and verify it was auto-assigned
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID)
VALUES ('Lina', 'Alharthy', 'F', 27, 'lina@gmail.com', 'pass123', 1);
COMMIT;
SELECT emp_ID, fname, lname, emp_email
FROM EMPLOYEE
WHERE emp_email = 'lina@gmail.com';

--- HANDLE ERROR
SELECT MAX(emp_ID) FROM EMPLOYEE;
DROP SEQUENCE EMP_SEQ;

CREATE SEQUENCE EMP_SEQ

```

Below the SQL editor, the 'Query Result' tab is active, showing the result of the SELECT statement:

EMP_ID	FNAME	LNAME	EMP_EMAIL
1	12 Lina	Alharthy	lina@gmail.com

TRG_EMP_WELCOME_LOG

An AFTER INSERT trigger records every new employee insertion.

Result

Whenever a new employee is added, a corresponding log entry with action:

NEW HIRE

is inserted into EMPLOYEE_LOG.

Worksheet

Query Builder

```

FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEE_LOG (
        log_id,
        emp_ID,
        action,
        log_timestamp
    )
    VALUES (
        EMPLOYEE_LOG_SEQ.NEXTVAL,
        :NEW.emp_ID,
        'NEW HIRE',
        SYSTIMESTAMP
    );
END;
/
--Test by inserting 2 employees and querying EMPLOYEE_LOG
INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID)
VALUES ('Mona', 'Alharthi', 'F', 25, 'mona@gmail.com', 'pass123', 1);

INSERT INTO EMPLOYEE (fname, lname, gender, age, emp_email, emp_pass, job_ID)
VALUES ('Salim', 'Albalushi', 'M', 30, 'salim@gmail.com', 'pass123', 2);

COMMIT;

SELECT *
FROM EMPLOYEE_LOG
ORDER BY log_id;

```

Script Output x

Query Result x

	LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	1		13 NEW HIRE	17-JUN-26 06.58.02.097000000 PM
2	2		14 NEW HIRE	17-JUN-26 06.58.02.105000000 PM

19. Scheduler Jobs

JOB_GREET_EMPLOYEES

A one-time scheduler job was created to:

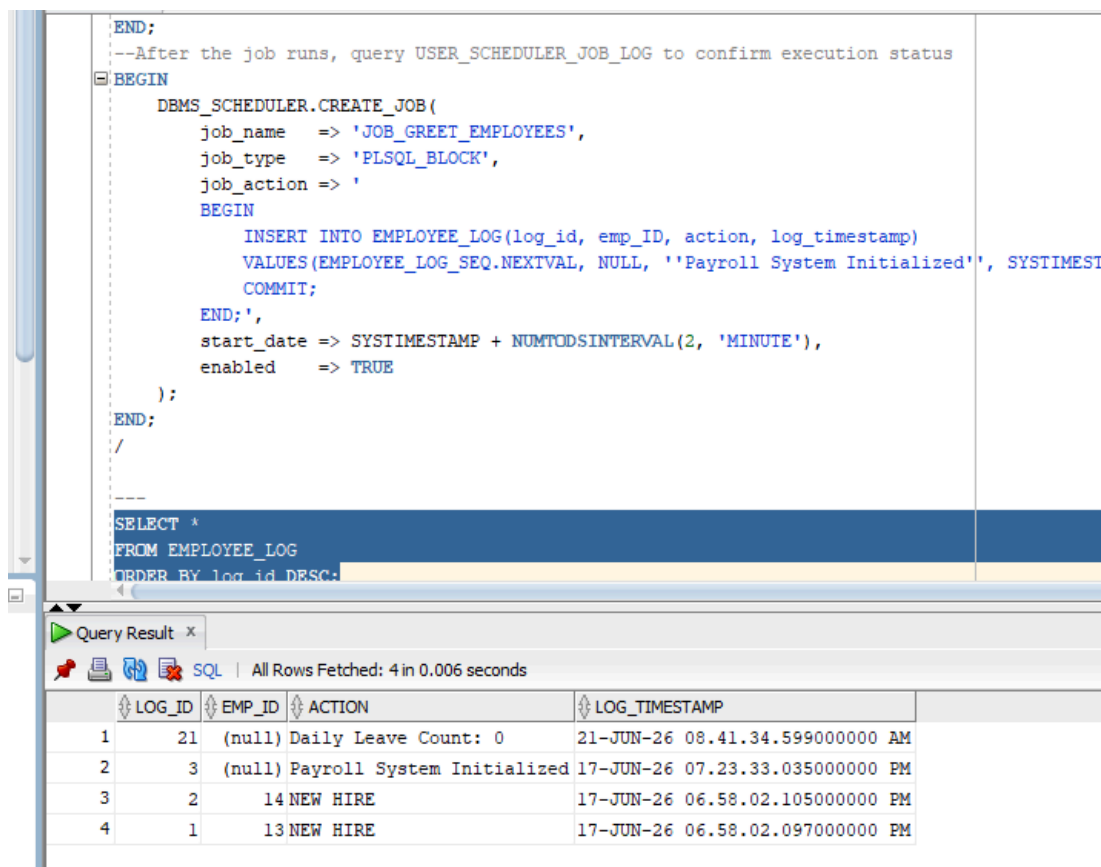
- Insert a log entry.
- Record system initialization.

Result

The following entry was successfully added:

Payroll System Initialized

The scheduler job executed successfully after two minutes and inserted a log record into EMPLOYEE_LOG with the action "Payroll System Initialized", confirming successful execution of the scheduled task.



The screenshot displays the SQL Developer interface. The top pane shows a PL/SQL block that creates a scheduler job named 'JOB_GREET_EMPLOYEES'. The job is configured to run every 2 minutes and execute a block of SQL that inserts a log record into the 'EMPLOYEE_LOG' table. The log record contains the action 'Payroll System Initialized'. The bottom pane shows the query result for the SQL statement executed by the job, which is a SELECT query from the 'EMPLOYEE_LOG' table ordered by log_id in descending order. The result shows four rows of log data.

```
END;
--After the job runs, query USER_SCHEDULER_JOB_LOG to confirm execution status
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name => 'JOB_GREET_EMPLOYEES',
    job_type => 'PLSQL_BLOCK',
    job_action => '
    BEGIN
      INSERT INTO EMPLOYEE_LOG(log_id, emp_ID, action, log_timestamp)
      VALUES(EMPLOYEE_LOG_SEQ.NEXTVAL, NULL, 'Payroll System Initialized', SYSTIMEST
      COMMIT;
    END;',
    start_date => SYSTIMESTAMP + NUMTODSINTERVAL(2, 'MINUTE'),
    enabled => TRUE
  );
END;
/

SELECT *
FROM EMPLOYEE_LOG
ORDER BY log_id DESC;
```

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	21	(null) Daily Leave Count: 0	21-JUN-26 08.41.34.599000000 AM
2	3	(null) Payroll System Initialized	17-JUN-26 07.23.33.035000000 PM
3	2	14 NEW HIRE	17-JUN-26 06.58.02.105000000 PM
4	1	13 NEW HIRE	17-JUN-26 06.58.02.097000000 PM

JOB_DAILY_LEAVE_REPORT

A recurring scheduler job was created to:

- Run daily at 07:00 AM.
- Count leave records.

- Insert a summary into EMPLOYEE_LOG.

Schedule

The screenshot displays the Oracle SQL Developer interface. The top pane shows a PL/SQL script for creating a job. The bottom pane shows the query results for the job's configuration.

Script:

```

v_count NUMBER;
BEGIN
  SELECT COUNT(*)
  INTO v_count
  FROM LEAVE
  WHERE TRUNC(leave_date) = TRUNC(SYSDATE);

  INSERT INTO EMPLOYEE_LOG(log_id, emp_ID, action, log_timestamp)
  VALUES (EMPLOYEE_LOG_SEQ.NEXTVAL, NULL, 'Daily Leave Count: ' || v_count, SYSTIMESTAMP);

  COMMIT;
END;
/

```

Query Results:

JOB_NAME	JOB_SUBNAME	JOB_STYLE	JOB_CREATOR	CLIENT_ID	GLOBAL_UID	PROGRAM_OWNER	PROGRAM_NAME	JOB_TYPE	JOB_ACTION
JOB_DAILY_LEAVE_REPORT	(null)	REGULAR	EMPLOYEEMANAGEMENTSYSTEM	(null)	(null)	(null)	(null)	PLSQL_BLOCK	DECLARE v_count NUMBER; BEGIN SELECT COUNT(*) INTO v_count FROM LEAVE WHERE TRUNC(leave_date) = TRUNC(SYSDATE); INSERT INTO EMPLOYEE_LOG(log_id, emp_ID, action, log_timestamp) VALUES (EMPLOYEE_LOG_SEQ.NEXTVAL, NULL, 'Daily Leave Count: ' v_count, SYSTIMESTAMP); COMMIT; END; /

Script:

```

SELECT job_name, enabled, repeat_interval, state
FROM USER_SCHEDULER_JOBS
WHERE JOB_NAME = 'JOB_DAILY_LEAVE_REPORT';

```

Query Results:

JOB_NAME	ENABLED	REPEAT_INTERVAL	STATE
JOB_DAILY_LEAVE_REPORT	TRUE	FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0	SCHEDULED

Conclusion

The Employee Management System was successfully implemented using Oracle SQL and PL/SQL. The project demonstrated database design, data manipulation, aggregation, joins, subqueries, views, stored procedures, functions, triggers, and scheduler automation. All business requirements were implemented and tested successfully while maintaining data integrity and enforcing validation rules throughout the system.